

UNIVERSITÉ DE TECHNOLOGIE DE TROYES

TP Docker — 2026

Mini projet noté

Conteneuriser une application avec docker

Pierre GUILLOU

5 octobre 2026

Rapport

Table des matières

1	Contexte et objectifs	2
1.1	Le besoin exprimé	2
1.2	Travail demandé	2
2	Choix de l'image de base	2
3	Cache	3
3.1	Explication de l'ordre des instructions	3
3.2	Mesures	3
3.3	Affichage des résultats	4
4	Taille de l'image	4
5	Persistance	5
6	Difficultés	6

1 Contexte et objectifs

1.1 Le besoin exprimé

Pas votre capacité à écrire du code applicatif. Votre capacité à rendre une application démarable par quelqu'un d'autre, sur une machine qu'il n'a pas préparée, en une commande. C'est exactement la compétence attendue en entreprise lorsque vous arrivez sur un projet et qu'on vous demande de le faire tourner en local.

1.2 Travail demandé

Reprenez un projet que vous avez déjà écrit dans un autre module, ou un projet personnel : API REST, application Django, Flask ou FastAPI, back Node/Express, application Spring Boot, site PHP — le langage est indifférent. Le code peut être modeste : il n'est pas noté. Une seule contrainte : ce ne peut pas être le TP 4 recopié. Changez au minimum le langage ou le moteur de base de données, et ajoutez une fonctionnalité qui vous est propre.

Les livrables attendus sont de la forme suivante :

```
votre-projet/
├── compose.yaml
├── Dockerfile
│   └── (ou plusieurs, un par service construit)
├── .dockerignore
├── .env.example
│   └── commité, avec des valeurs d'exemple
├── .gitignore
│   └── contient au minimum .env
├── README.md
│   └── voir paragraphe 6
├── RAPPORT.md
│   └── voir paragraphe 7
└── src/ (ou app/, back/, ...)
    └── votre code
```

2 Choix de l'image de base

J'ai retenu node :24-slim pour l'application. Node 24 est la version LTS (support long terme), et le tag est figé sur cette version, contrairement à latest qui peut changer d'un jour à l'autre.

La variante complète node :24 repose sur une Debian complète qui embarque des compilateurs (gcc, make, python), des outils de développement et de la documentation. Ces outils ne servent qu'à compiler des modules natifs. Or mes deux dépendances, express et pg, sont écrites en JavaScript pur : la variante complète alourdirait l'image sans rien apporter.

La variante node :24-alpine est la plus légère, mais elle repose sur Alpine Linux, qui utilise la bibliothèque C musl au lieu de glibc. Certains modules natifs ou outils se comportent différemment avec musl, ce qui peut causer des erreurs difficiles à diagnostiquer. La variante slim garde une Debian standard (glibc, outils habituels) avec seulement le strict nécessaire pour exécuter Node : c'est un compromis entre taille et compatibilité.

Pour la base, j'utilise l'image officielle postgres :17. J'ai évité la version 18, dont l'image a changé le chemin de stockage des données, pour garder le chemin classique /var/lib/postgresql/data, où est monté mon volume.

3 Cache

Instructions : montrez l'ordre de vos instructions et expliquez ce qui est reconstruit quand vous modifiez une ligne de code. Collez la sortie de deux builds successifs avec leurs durées.

Pour répondre à cette consigne, nous suivrons le chemin de réflexion suivant :

1. explication de l'ordre des instructions
2. création d'un dossier mesures pour y stocker les résultats du build et des temps associés
3. execution

3.1 Explication de l'ordre des instructions

Lors de la construction de l'image avec Dockerfile, on va partir d'une image de base et ensuite empiler des couches, correspondant à des instructions exécutées dans le cache, pour au final obtenir une image optimisée au besoin.

L'ordre des instructions dans le Dockerfile impacte donc directement le cache. Il convient donc de placer les instructions qui changent rarement (installation de dépendances système) en haut, et celles qui changent souvent (copie du code source) en bas. Ainsi, le cache est réutilisé au maximum (source : <https://blog.stephane-robert.info/docs/conteneurs/images-conteneurs/>).

En appliquant cette logique à mon Dockerfile : Les fichiers de dépendances (package.json et package-lock.json) changent rarement : ils sont copiés seuls, puis npm ci installe les dépendances, ce qui est l'étape la plus longue. Le code, qui change souvent, n'est copié qu'ensuite, par COPY.

Quand je modifie une ligne de server.js, les fichiers de dépendances sont inchangés : les couches jusqu'à RUN npm ci sont reprises du cache, marquées CACHED, et seule COPY . . est refaite, ainsi que les instructions suivantes. Si j'avais copié tout le code avant d'installer les dépendances, la moindre modification du code aurait invalidé le cache et relancé npm ci à chaque build.

3.2 Mesures

Pour effectuer les mesures, j'ai créé un dossier dédié (`mkdir mesures`) dans lequel sont enregistrés les résultats des différents builds.

```
1 time docker compose --progress plain build --no-cache web 2>&1 | tee
  mesures/build1.txt
```

Le premier build est réalisé sans utiliser le cache Docker. Ses résultats sont enregistrés dans le fichier `mesures/build1.txt`.

```
1 echo "// test du cache" >> app/server.js
```

Cette commande modifie le fichier `app/server.js` afin de tester l'utilisation du cache Docker.

```
1 time docker compose --progress plain build web 2>&1 | tee mesures/build2.txt
```

Le second build utilise le cache Docker lorsque cela est possible. Ses résultats sont enregistrés dans le fichier `mesures/build2.txt`.

3.3 Affichage des résultats

TABLE 1 – Ordre des instructions et comportement du cache après modification d’une ligne de `server.js`

Étape	Instruction	Fréquence de modification	Build 1	Build 2
1/5	FROM node:24-slim	jamais	0.1 s	0.0 s
2/5	WORKDIR /app	jamais	0.0 s	CACHED
3/5	COPY package.json package-lock.json ./	rarement	0.1 s	CACHED
4/5	RUN npm ci --omit=dev	rarement	7.5 s	CACHED
5/5	COPY . .	à chaque modification du code	0.1 s	0.0 s
—	Export de l’image	—	2.6 s	0.2 s

Instructions de configuration non numérotées : ENV, USER, EXPOSE et CMD.

TABLE 2 – Durée des deux builds successifs

Build	Contexte	Durée totale
1	--no-cache : les dépendances sont réinstallées avec <code>npm ci --omit=dev</code>	10.5 s
2	Modification d’une ligne de <code>server.js</code> ; la couche <code>npm ci --omit=dev</code> est réutilisée depuis le cache	0.2 s
Gain de temps observé		environ $\times 50$

Remarque : L’indication CACHED signifie que l’instruction n’a pas été exécutée mais que les données caches ont été réutilisées.

4 Taille de l’image

Mon image *bebe_suivi-web*, construite sur `node:24-slim`, pèse **343 Mo** sur le disque (colonne `DISK USAGE` de la commande `docker image ls`) et **83 Mo** une fois compressée (colonne `CONTENT SIZE`, correspondant à ce qui serait téléchargé depuis un registre).

Ces deux valeurs sont cohérentes avec `docker history`, qui donne la taille décompressée de chaque couche : leur somme atteint 259,6 Mo, et $259,6 + 83 \approx 343$ Mo. Avec le stockage d’images Docker utilisé, la colonne `DISK USAGE` compte en effet à la fois les couches décompressées et les archives compressées conservées sur le disque.

Pour mesurer l’optimisation, j’ai construit la même image avec la variante complète `node:24`, sans rien changer d’autre au Dockerfile.

TABLE 3 – Taille de l’image avant et après optimisation

Mesure	Avant : node:24	Après : node:24-slim	Gain
Sur le disque (<code>DISK USAGE</code>)	1,64 Go	343 Mo	-1 297 Mo (-79 %)
Compressée (<code>CONTENT SIZE</code>)	412 Mo	83 Mo	-329 Mo (-80 %)
Décompressée (somme de <code>docker history</code>)	1 232 Mo	260 Mo	-972 Mo (-79 %)

L’image est environ cinq fois plus légère, quelle que soit la mesure retenue. Le tableau suivant, tiré de `docker history`, montre d’où vient ce gain.

TABLE 4 – Poids des couches selon l'image de base (tailles décompressées)

Contenu des couches	node:24	node:24-slim
Mon application : COPY . .	45,1 ko	45,1 ko
Mes dépendances : RUN <code>npm ci --omit=dev</code>	10,8 Mo	10,8 Mo
COPY <code>package.json package-lock.json ./</code> et WORKDIR	57,4 ko	57,4 ko
Node.js 24.21.0	217 Mo	156 Mo
Yarn et script de démarrage	5,4 Mo	7,4 Mo
Paquets de développement (compilateurs, bibliothèques)	619 Mo	–
Outils de gestion de versions (<code>git</code> , etc.)	194 Mo	–
Outils réseau (<code>curl</code> , etc.)	52,3 Mo	–
Système Debian 12 « bookworm »	133 Mo	85,3 Mo
Total	1 232 Mo	260 Mo

Le gain vient **uniquement de l'image de base**. Les couches produites par mon Dockerfile sont strictement identiques dans les deux versions : mon code pèse 45 ko et mes 82 dépendances pèsent 10,8 Mo, soit environ 4 % de l'image finale. En revanche, la variante complète ajoute trois couches absentes de `slim`, qui pèsent à elles seules 865 Mo : des compilateurs et bibliothèques de développement, des outils de gestion de versions et des outils réseau.

La variante `slim` part également d'un système Debian réduit (85,3 Mo contre 133 Mo) et d'une installation de Node.js allégée (156 Mo contre 217 Mo).

Deux autres choix évitent d'alourdir l'image, avec un effet plus modeste : `npm ci --omit=dev` n'installe pas les dépendances de développement, et le fichier `.dockerignore` exclut du contexte de build `node_modules`, `.git` et `.env`.

5 Persistance

Les données de PostgreSQL sont écrites dans `/var/lib/postgresql/data`, où est monté le volume nommé `db_data` (nom réel : `bebe_suivi_db_data`). Ce volume est géré par Docker, indépendamment des conteneurs : supprimer un conteneur ne supprime pas son volume.

Lors d'un `docker compose down` puis `up` : `down` arrête et supprime les trois conteneurs (`web`, `db` et `adminer`) ainsi que le réseau `app-network`, mais conserve le volume. Au `up`, Compose recrée des conteneurs neufs, et le nouveau conteneur `db` remonte le même volume, dont la date de création n'a pas changé. Au démarrage, l'image `postgres` constate que le dossier de données contient déjà une base : elle ne réinitialise rien, ignore les variables `POSTGRES_*` et n'exécute pas `db/init.sql`. La table `events` et toutes ses lignes sont retrouvées telles quelles. Le service `web` attend que `db` soit `healthy`, puis se reconnecte à la base par son nom.

Lors d'un `docker compose down -v` : l'option `-v` supprime en plus les volumes nommés déclarés dans `compose.yaml`, donc `db_data` et toutes les données qu'il contient. Au `up` suivant, Docker crée un nouveau volume vide, avec une nouvelle date de création. PostgreSQL trouve un dossier vide : il initialise une base neuve, crée l'utilisateur et la base à partir des variables du `.env`, puis exécute `db/init.sql`.

La structure globale des fichiers étant plus claire, l'étape suivante est la création de la table SQL. Elle comporte 27 colonnes : 2 colonnes supplémentaire pour spécifier de quel fichier et de quelle ligne provient chaque donnée, 5 colonnes décrivant le ticket, puis 5 blocs de 4 colonnes pour les articles.

6 Difficultés

Le symptôme. Au premier lancement complet de l'application, les trois conteneurs démarraient normalement et `db` était `healthy`. J'ai testé l'API avec cinq requêtes successives : vérification de la connexion, création d'une tétée, création d'une couche, tétée invalide et lecture des événements. Seule la première fonctionnait :

```
{"status":"ok"}{"error":"Erreur interne"}{"error":"Erreur interne"}
{"error":"Erreur interne"}{"error":"Erreur interne"}
```

La connexion à la base fonctionnait car j'ai réussi à me connecter depuis la page d'administrer (`http://localhost:8081/`). Le réseau, le nom de service `db` et les identifiants n'étaient pas en cause. Cette route n'exécute que `SELECT 1` dans le `server.js`, qui n'a besoin d'aucune table. Toutes les autres routes, y compris la simple lecture, interrogent la table `events` : le problème venait donc de cette table.

Le message exact. Le message « Erreur interne » est volontairement générique côté client : le gestionnaire d'erreurs de `server.js` renvoie une erreur 500 sans détail, mais écrit l'erreur complète dans les logs. Je les ai donc consultés (extrait) :

```
$ docker compose logs web
web-1 | Bébé suivi en écoute sur le port 8080
web-1 | error: relation "events" does not exist
web-1 |       at /app/node_modules/pg-pool/index.js:45:11
web-1 |       at process.processTicksAndRejections (node:internal/...)
web-1 |       at async /app/server.js:39:20 {
web-1 |   severity: 'ERROR',
web-1 |   code: '42P01',
web-1 |   position: '15',
web-1 |   [...]
web-1 |   routine: 'parserOpenTable'
web-1 | }
```

Ces quelques lignes suffisent à localiser le problème :

- `relation "events" does not exist` : PostgreSQL ne trouve pas la table `events` (en PostgreSQL, « relation » désigne une table) ;
- `code: '42P01'` est le code standard SQL `undefined_table` : il ne s'agit ni d'un problème de connexion, ni d'un problème de droits ;
- `server.js:39` est la ligne de la route `GET /api/events` qui exécute `SELECT * FROM events ...` ;
- `position: '15'` désigne le quinzième caractère de cette requête, soit exactement le début du mot `events`.

La base, elle, existait, mais elle ne contenait aucune table :

```
$ docker compose exec db psql -U baby -d babytracker -c "\dt"
\ac
```

Une première hypothèse, réfutée. Mon script `db/init.sql`, qui crée la table, n'avait donc pas été exécuté. Je savais que l'image `postgres` n'exécute ses scripts d'initialisation que si le volume de données est vide. J'ai d'abord supposé que le volume contenait déjà une base créée lors d'un démarrage antérieur, et j'ai donc repris d'un volume vide :

```
$ docker compose down -v
$ docker compose up -d
```

Les erreurs ont persisté à l'identique. Le volume était donc bien vide : PostgreSQL avait initialisé

une base neuve, mais sans exécuter mon script. Le problème n'était pas *quand* le script s'exécutait, mais *si* PostgreSQL le voyait.

La cause. J'ai regardé le contenu du dossier d'initialisation à l'intérieur du conteneur :

```
$ docker compose exec db ls /docker-entrypoint-initdb.d/  
\ac
```

Le dossier était vide. En relisant mon `compose.yaml`, j'ai constaté que le service `db` ne montait que le volume de données. J'avais oublié de monter mon script dans ce dossier : il existait sur ma machine, mais pas dans le conteneur.

La correction. J'ai ajouté le montage du script, en lecture seule :

```
volumes:  
  - db_data:/var/lib/postgresql/data  
  - ./db/init.sql:/docker-entrypoint-initdb.d/init.sql:ro
```

puis relancé `docker compose down -v` et `docker compose up -d`. Les logs de PostgreSQL confirment cette fois l'exécution du script :

```
$ docker compose logs db | grep init.sql  
\ac
```

et toutes les routes de l'API fonctionnent.

Ce que j'en retiens. Un fichier présent dans mon projet n'existe pas pour un conteneur tant qu'il n'y est pas monté ou copié : le conteneur ne voit que ce que `compose.yaml` ou le Dockerfile lui donnent explicitement. Je retiens aussi la démarche : isoler ce qui fonctionne (la connexion) de ce qui échoue (les requêtes sur la table), lire le message complet dans les logs plutôt que l'erreur générique, et tester une hypothèse avant de la considérer comme acquise. C'est l'échec du `down -v` qui m'a orienté vers la vraie cause.

Annexes

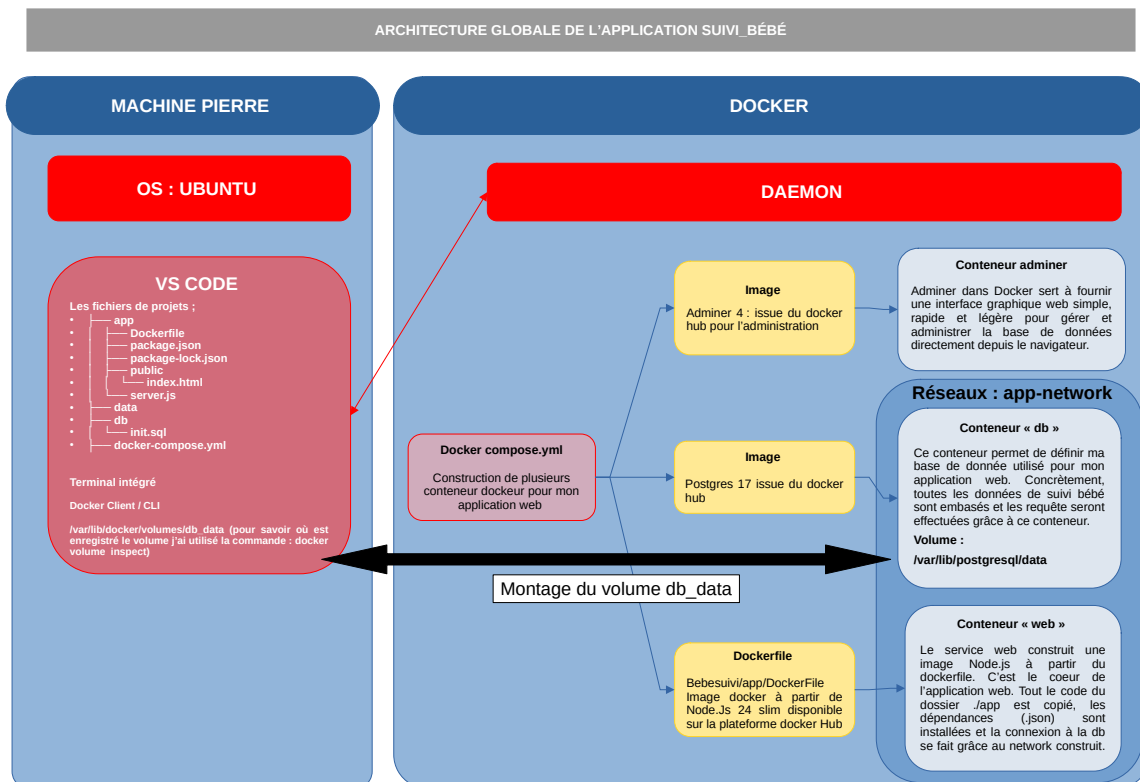


FIGURE 1 – Architecture globale des conteneurs et du réseau app-network

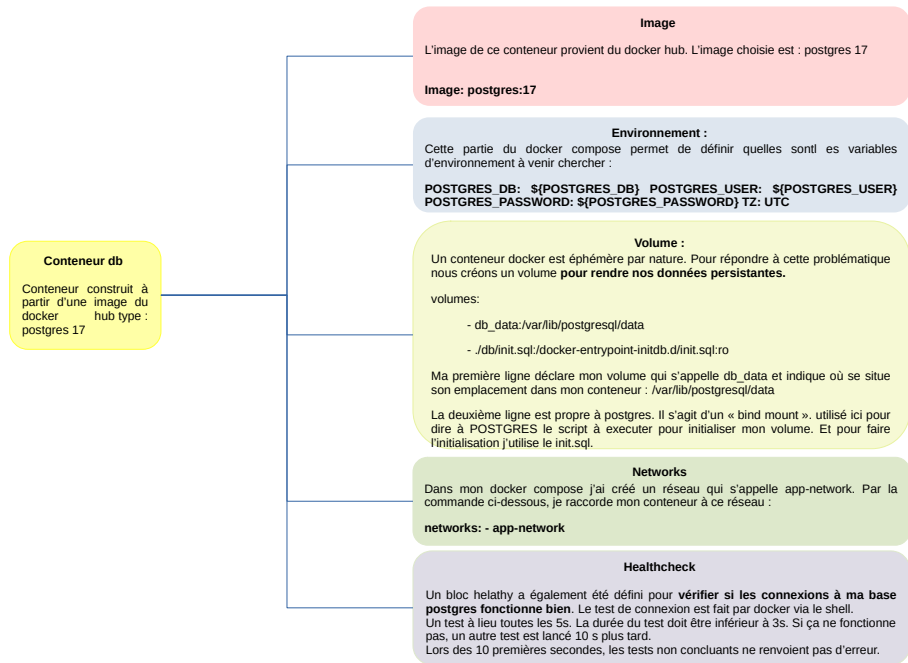


FIGURE 2 – Architecture du conteneur db

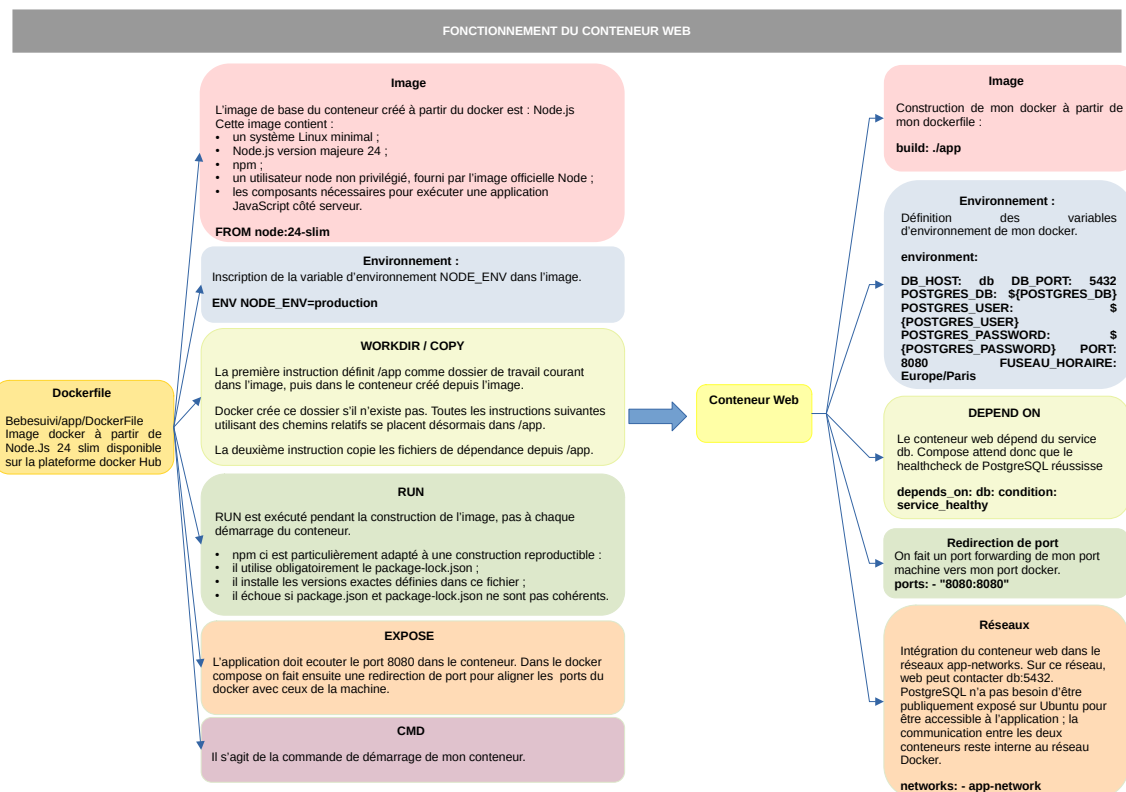


FIGURE 3 – Architecture du conteneur web